

B4.1.

Activitat repàs LAMP i Intro J2EE.

Marta Llabrés Roldós

M8 Desplegament d'aplicacions web

2n Daw

Curs 25/26

Index

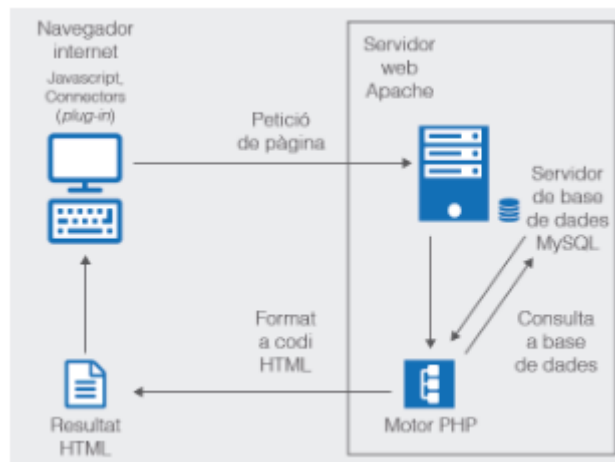
Reflexió sobre el que estem fent (individual)	3
1. Explicar	3
1.1 Com circula una petició HTTP en LAMP (esquema). Argumentar quin paper té Apache, quin PHP, quin MySQL.	3
1.2 Avantatges de LAMP i limitacions si el sistema creix.	3
2. Respon les següents qüestions argumentant les respostes	4
2.1 Què aporta un servidor d'aplicacions que no aporta un servidor web?	4
2.2 Per què separar frontend i backend pot millorar escalabilitat?	4
2.3 En quins casos una arquitectura LAMP és suficient?	5
2.4 En quins casos recomanaríeu Tomcat o un servidor J2EE?	5
2.5 Què és Catalina, Coyote i Jasper. Quin paper juguen.	5
Tasques pràctiques (en grup)	6
1. Instal·lació del JDK	6
2. Descàrrega de Tomcat	7
3. Arrencada de Tomcat	8
4. Accedir localhost i explicar el flux Catalina -> Coyote -> resposta en aquest cas concret.	8
5. Identificar carpetes: /bin /conf /webapps /logs /work	9
6. Mira el contingut d'aquestes carpetes i fes una taula que descrigui la funció de la carpeta i els fitxers més rellevants que contenen.	10
7. Anàlisi de conf/server.xml i conf/web.xml	10

Reflexió sobre el que estem fent (individual)

1. Explicar

1.1 Com circula una petició HTTP en LAMP (esquema). Argumentar quin paper té Apache, quin PHP, quin MySQL.

- Circulació d'una petició HTTP a LAMP



Tal i com es mostra a l'esquema, des del navegador es realitza una petició que s'envia al servidor web apache, dins del qual el motor PHP es comunica amb el servidor de base de dades per tal de fer la consulta a aquesta i un cop obtingudes les dades, s'envia la resposta en format HTML i es mostra al navegador.

- Paper d'Apache, PHP i MySql

Apache el podem considerar com un porter ja que és l'encarregat de rebre les peticions HTTP dels navegadors i decideix com gestionar-les.

PHP és el que s'encarrega d'executar la lògica de l'aplicació ja que pot processar formularis, gestiona sessions i és el que finalment construeix l'HTML dinàmic que es retornarà al navegador. Necessita Apache per rebre les peticions i MySQL per obtenir dades.

MySQL el podem considerar com un magatzem ja que és l'encarregat de guardar totes les dades persistents (usuaris, productes, comandes...). No es comunica directament amb el navegador mai, només respon consultes SQL que li fa PHP.

1.2 Avantatges de LAMP i limitacions si el sistema creix.

- Avantatges

LAMP és un sistema d'infraestructura d'internet que utilitza Linux (SSOO), Apache (servidor web), MySQL/MariaDB (gestor de base de dades) i Perl/PHP/Python (llenguatge de programació).

El fet que aquest tipus de infraestructura compti amb aquestes eines permet als professionals de la informàtica alleugerir la feina de manteniment del sistema, al ser de codi obert és disposa de software gratuït i amb documentació disponible i, a més, al ser una de les arquitectures més usades passa a ser una de les més esteses amb estabilitat i seguretat constatada.

- Limitacions quan el sistema creix:

En el cas de creixement del sistema és molt probable que ens trobem amb limitacions i falles del sistema, i és per això que és important configurar un conjunt de servidors per a que les peticions dels usuaris es distribueixin entre aquests servidors i així poder aconseguir:

1. Una alta disponibilitat, és a dir, en el cas que un servidor caigui, disposar d'un altre servidor per prestar el servei
2. Disposar d'un balanç de càrrega que permeti a cada servidor atendre un percentatge de les peticions, de tal forma que el sistema en conjunt pugui admetre més usuaris.

2. Respon les següents qüestions argumentant les respostes

2.1 Què aporta un servidor d'aplicacions que no aporta un servidor web?

- Servei d'aplicacions:

Un servidor d'aplicacions és un programa que ofereix un entorn d'execució per a aplicacions i poden gestionar serveis d'autenticació, implementació de la lògica de negoci, gestió de sessions d'usuari, accés a components o llibreries, cues de missatges i connexions de bases de dades.

El principal benefici d'aquest tipus de servidors és que el desenvolupador no ha de programar des de zero funcionalitats com autenticació o connexions a bases de dades, sinó que les pot utilitzar directament com a blocs ja construïts que ofereix el propi servidor.

- Servidor web:

Un servidor web és un tipus de programari que rep peticions HTTP/HTTPS dels navegadors i retorna respostes en fitxers estàtics com podria ser un fitxer HTML, CSS o imatges.

2.2 Per què separar frontend i backend pot millorar escalabilitat?

El fet que frontend i backend estiguin separats permet que cada part pot escalar i millorar de forma independent segons les necessitats. Si hi ha moltes peticions al backend (per exemple, càlculs pesats o consultes a la base de dades), es poden afegir més servidors sense tenir la necessitat de tocar el frontend, i viceversa. A més, permet distribuir la càrrega de manera més eficient, ja que el servidor de frontend pot servir contingut estàtic des de cachés o CDNs, mentre el backend es concentra únicament en la lògica de negoci.

2.3 En quins casos una arquitectura LAMP és suficient?

Principalment, LAMP és una infraestructura adequada per a aplicacions web dinàmiques amb PHP sense necessitat de Java EE

2.4 En quins casos recomanaríeu Tomcat o un servidor J2EE?

- J2EE

J2EE és un tipus de servidor que resultaria molt recomanat per a aplicacions més complexes que necessiten serveis més avançats com transaccions, cues de missatges o la gestió de múltiples aplicacions amb alta disponibilitat.

- TOMCAT

Tomcat és un contenidor web basat en el llenguatge Java, per tant seria molt recomanat usar-lo per aplicacions web Java relativament senzilles que fan servir servlets i JSP, projectes petits o mitjans, APIs REST o quan es vol un servidor lleuger.

2.5 Què és Catalina, Coyote i Jasper. Quin paper juguen.

Catalina és el contenidor de servlets, motor principal de Tomcat i amb el que es poden configurar sistemes de control d'accés definint usuaris, contrasenyes i rols per protegir l'aplicació.

Coyote és un connector que admet peticions HTTP1.1 i permet a Tomcat actuar com un servidor web.

Jasper és l'encarregat de compila fitxers JSP de manera que es puguin comportar com a servlets dirigits per Catalina.

Tasques pràctiques (en grup)

1. Instal·lació del JDK

Abans d'instal·lar Tomcat cal disposar d'un entorn Java al sistema. Tomcat 10.x requereix com a mínim Java 11. En sistemes basats en Debian/Ubuntu, s'ha de procedir de la manera següent:

1. S'ha d'actualitzar la llista de paquets del sistema:

\$ sudo apt update

2. A continuació, s'installa el JDK (Java Development Kit). Es recomana la versió 17 per la seva compatibilitat amb Tomcat 10 i el seu suport a llarg termini (LTS):

\$ sudo apt install openjdk-17-jdk -y

```
mar@meow:~$ sudo apt install openjdk-17-jdk -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
```

3. Per verificar que la instal·lació s'ha completat correctament, s'ha d'executar la comanda següent. La sortida hauria de mostrar la versió instal·lada:

\$ java -version

```
mar@meow:~$ java -version
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)
```

4. Si existeixen múltiples versions de Java instal·lades, cal assegurar-se que la versió activa és la correcta. Per comprovar-ho i canviar-la si cal:

\$ sudo update-alternatives --config java

S'ha de seleccionar la versió 17 de la llista que apareix en pantalla.

```
mar@meow:~$ sudo update-alternatives --config java
Existen 3 opciones para la alternativa java (que provee /usr/bin/java).



| Selección | Ruta                                        | Prioridad | Estado          |
|-----------|---------------------------------------------|-----------|-----------------|
| 0         | /usr/lib/jvm/jdk-25-oracle-x64/bin/java     | 419430400 | modo automático |
| * 1       | /usr/lib/jvm/java-17-openjdk-amd64/bin/java | 1711      | modo manual     |
| 2         | /usr/lib/jvm/java-25-openjdk-amd64/bin/java | 2511      | modo manual     |
| 3         | /usr/lib/jvm/jdk-25-oracle-x64/bin/java     | 419430400 | modo manual     |



Pulse <Intro> para mantener el valor por omisión [*] o pulse un número de selección: 1
```

5. Finalment, és convenient definir la variable d'entorn JAVA_HOME, que Tomcat utilitzarà per localitzar el JDK. S'ha d'afegir la línia següent al fitxer /etc/environment:

JAVA_HOME="/usr/lib/jvm/java-17-openjdk-amd64"

```
GNU nano 7.2 /etc/environment *
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:
JAVA_HOME="/usr/lib/jvm/java-17-openjdk-amd64"
```

Per aplicar el canvi sense reiniciar la sessió:

```
$ source /etc/environment
```

```
$ echo $JAVA_HOME
```

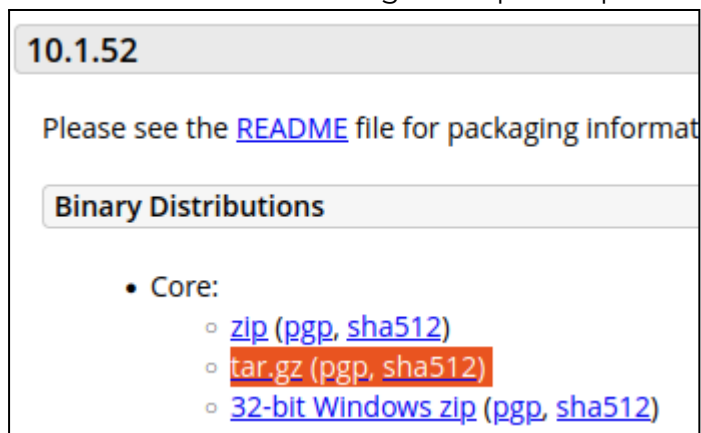
```
mar@meow:~$ source /etc/environment
mar@meow:~$ echo $JAVA_HOME
/usr/lib/jvm/java-17-openjdk-amd64
```

2. Descàrrega de Tomcat

La descàrrega de Tomcat es fa des del lloc web oficial del projecte Apache. S'ha d'accedir a la URL:

<https://tomcat.apache.org/download-10.cgi>

Des d'aquesta pàgina, a la secció Binary Distributions, s'ha de descarregar l'arxiu comprimit en format .tar.gz corresponent a la versió estable més recent per a Linux. El nom de l'arxiu seguirà el patró apache-tomcat-10.x.x.tar.gz.



Un cop descarregat, s'ha d'extreure l'arxiu i moure el directori resultant a /opt, que és la ubicació convencional per a programari de tercers en sistemes Linux:

```
$ tar -xzf apache-tomcat-10.1.x.tar.gz
```

```
$ sudo mv apache-tomcat-10.1.x /opt/tomcat
```

```
mar@meow:~/Descargas$ tar -xzf apache-tomcat-10.1.52.tar.gz apache-tomcat-10.1.52/
mar@meow:~/Descargas$ sudo mv apache-tomcat-10.1.52 /opt/tomcat
```

Per qüestions de seguretat, es recomana crear un usuari del sistema dedicat per a Tomcat, de manera que el servidor no s'executi amb privilegis de superusuari:

```
$ sudo useradd -m -U -d /opt/tomcat -s /bin/false tomcat
```

```
mar@meow:~$ sudo useradd -m -U -d /opt/tomcat -s /bin/false tomcat
useradd: warning: the home directory /opt/tomcat already exists.
useradd: Not copying any file from skel directory into it.
mar@meow:~$
```

```
$ sudo chown -R tomcat: /opt/tomcat
```

```
mar@meow:~$ sudo chown -R tomcat: /opt/tomcat
mar@meow:~$
```

3. Arrencada de Tomcat

Per iniciar Tomcat, s'ha d'executar el script startup.sh que es troba a la carpeta bin del directori d'instal·lació:

```
$ sudo -u tomcat /opt/tomcat/bin/startup.sh
```

```
mar@meow:~$ sudo -u tomcat /opt/tomcat/bin/startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr/lib/jvm/java-17-openjdk-amd64
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Tomcat started.
```

Per verificar que el procés s'està executant correctament, es pot comprovar que el port 8080 està a l'escolta:

```
$ ss -tlnp | grep 8080
```

```
mar@meow:~$ ss -tlnp | grep 8080
LISTEN 0      100          *:8080      *:*
```

4. Accedir localhost i explicar el flux Catalina → Coyote → resposta en aquest cas concret.

Al tenir instal·lat tant JDK com Tomcat ens permet accedir a <http://localhost:8080>. En aquest cas, el flux de treball entre Catalina, Coyote i la resposta que rebem és:

1. Coyote (Connector HTTP) Coyote és el component que escolta el port **8080**. Quan el teu navegador fa la petició HTTP, Coyote la rep, la processa a nivell de protocol HTTP i la transforma en un objecte `HttpServletRequest` que Tomcat pot entendre.

2. Catalina (Contenedor de Servlets) Catalina rep la petició que ha enviat Coyote i s'encarrega de fer 2 coses:

- Primer ha de buscar quin **Engine → Host → Context** ha de gestionar aquesta petició (en aquest cas l'aplicació per defecte `ROOT`)

- Segon ha d'invocar el **Servlet** corresponent, que en el cas de `localhost:8080` és el servlet de la pàgina de benvinguda de Tomcat (els fitxers estàtics que hi ha a `/var/lib/tomcat10/webapps/ROOT/`)

3. Resposta Un cop el Servlet genera la resposta, en aquest cas l'HTML de la pàgina de benvinguda, el camí torna enrere: Catalina empaqueta la resposta en un `HttpServletResponse` i Coyote la converteix de nou a protocol HTTP i l'envia pel port 8080 al navegador

Esquema resumit:



5. Identificar carpetes: `/bin` `/conf` `/webapps` `/logs` `/work`

Per trobar on són aquestes carpetes de Tomcat10 a Linux Mint farem servir aquestes comandes més directes:

`ls /usr/share/tomcat10/`

On amb aquesta comanda ja ens trobem amb la carpeta `/bin`

```
marta@marta-Linux-Mint:~$ ls /usr/share/tomcat10/
bin default.template etc lib logrotate.template
```

`ls /var/lib/tomcat10/`

On amb aquesta comanda ja ens trobem amb les carpetes `/conf`, `/logs`, `/webapps` i `/work`

```
marta@marta-Linux-Mint:~$ ls /var/lib/tomcat10/
conf lib logs policy webapps work
```

`ls /etc/tomcat10/`

On amb aquesta comanda ja ens trobem amb fitxers com `server.xml` i `web.xml`

```
marta@marta-Linux-Mint:~$ ls /etc/tomcat10/
Catalina catalina.properties context.xml jaspic-providers.xml l
ogging.properties policy.d server.xml tomcat-users.xml web.xml
```

El fet de trobar les carpetes i fitxes en localitzacions diferents és perquè `apt` segueix les convencions de Linux (FHS - Filesystem Hierarchy Standard):

- Configuració → `/etc`
- Dades → `/var/lib`

- Logs → `/var/log`
- Cache → `/var/cache`

6. Mira el contingut d'aquestes carpetes i fes una taula que descrigui la funció de la carpeta i els fitxers més rellevants que contenen.

Carpeta	Funcionalitat de la carpeta	Fitxers més rellevants
<code>/bin</code> <code>/usr/share/tomcat10/bin</code>	Conté els scripts d'inici i parada de Tomcat	<code>startup.sh</code> , <code>shutdown.sh</code> , <code>catalina.sh</code>
<code>/conf</code> <code>/etc/tomcat10/</code>	Conté tots els fitxers de configuració de Tomcat	<code>server.xml</code> , <code>web.xml</code> , <code>context.xml</code> , <code>tomcat-users.xml</code>
<code>/webapps</code> <code>/var/lib/tomcat10/webapps/</code>	Directorio on es despleguen les aplicacions web (WARs)	<code>ROOT/</code> (aplicació per defecte), fitxers <code>.war</code>
<code>/logs</code> <code>/var/log/tomcat10/</code>	Emmagatzema tots els registres d'activitat i errors	<code>catalina.out</code> , <code>localhost.log</code> , <code>access.log</code>
<code>/work</code> <code>/var/cache/tomcat10/work/</code>	Directorio de treball intern on Tomcat compila les JSP en Servlets	Fitxers <code>.java</code> i <code>.class</code> generats automàticament

7. Anàlisi de `conf/server.xml` i `conf/web.xml`

`conf/server.xml`

És el fitxer de configuració principal del servidor Tomcat. Defineix com funciona el servidor a nivell global: els ports on escolta, els protocols que utilitza, els hosts virtuals i quines aplicacions desplega i on. Sense aquest fitxer, Tomcat no sabria ni com arrencar.

Per tal de poder veure el contingut del fitxer executarem aquesta comanda a la terminal i així accedir-hi: `'sudo nano /etc/tomcat10/server.xml'`

```
GNU nano 7.2 /etc/tomcat10/server.xml
<?xml version="1.0" encoding="UTF-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements. See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License. You may obtain a copy of the License at
```

`conf/web.xml`

És el descriptor de desplegament global. No configura el servidor en si, sinó el comportament base de totes les aplicacions web que hi estan desplegades. Defineix quins servlets estan disponibles per defecte (com el `DefaultServlet` per a

fitxers estàtics o el JspServlet per a pàgines .jsp), els MIME types i quins fitxers es carreguen per defecte en accedir a un directori. Cada aplicació pot sobreescriure aquesta configuració amb el seu propi WEB-INF/web.xml.

Respon:

1. Quin port utilitza el connector HTTP?

Per a veure el port hem d'anar fins al <Connector> i veurem que tenim configurat el port 8080

```
<!-- A "Connector" represents an endpoint by which requests are received
and responses are returned. Documentation at :
HTTP Connector: /docs/config/http.html
AJP  Connector: /docs/config/ajp.html
Define a non-SSL/TLS HTTP/1.1 Connector on port 8080
-->
<Connector port="8080" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443"
    maxParameterCount="1000"
/>
```

2. Quin protocol està configurat?

A dins del mateix <Connector> podem veure que el protocol que tenim configurat és el "HTTP/1.1"

```
<Connector port="8080" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443"
    maxParameterCount="1000"
/>
```

3. On es defineix el context per defecte?

El context el podem trobar a dins del Host:

```
<Host name="localhost"  appBase="webapps"
    unpackWARs="true" autoDeploy="true">
```

En aquesta ocasió el paràmetre `appBase="webapps"` ens indica quina és la carpeta on Tomcat busca les aplicacions per defecte.

4. Quin rol té el fitxer web.xml global?

El fitxer `conf/web.xml` és el descriptor de desplegament global de Tomcat i s'aplica a totes les aplicacions web del servidor. Defineix els *servlets* per defecte: el *DefaultServlet*, que serveix fitxers estàtics (HTML, imatges, CSS, JS) quan no hi ha cap *servlet* específic per a la ruta, i el *JspServlet*, que gestiona les peticions a fitxers *.jsp* i *.jspx* delegant la compilació al motor *Jasper*. També defineix els *MIME types*, els *listeners del cicle de vida* i el *welcome-file-list*. El `WEB-INF/web.xml` de cada aplicació individual té prioritat sobre el global en cas de conflicte.